

## Code Explanation

### Delta Live Tables Implementation

This code is used to create a series of Delta Live Tables (DLT) for processing customer and order data. It demonstrates a multi-layered data processing pipeline, including bronze, silver, and gold layers, with Slowly Changing Dimension (SCD) Type 2 implementation.

#### ## Overview

The provided Python code is designed to work within the Databricks environment, utilizing the Delta Live Tables (DLT) framework to create a data pipeline. The pipeline processes customer and order data from CSV files, cleans and transforms the data, and then applies SCD Type 2 logic to track changes over time.

#### ## Step 1: Importing Necessary Libraries and Defining Bronze Tables

This step involves importing the required libraries and defining the bronze tables that read raw data from CSV files.

```
import dlt
from pyspark.sql.functions import col, lit, current_timestamp, when,
    expr
from pyspark.sql.types import DateType

@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from CSV files"
)
def customer_bronze():
    return spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customere
rdata")

@dlt.table(
    name="order_bronze",
    comment="Raw order data from CSV files"
)
def order_bronze():
    return spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderda
ta")
```

#### ## Step 2: Defining Silver Tables with Cleaned Data

In this step, the code defines silver tables that contain cleaned data. The data is cleaned by removing duplicates and null values.

```

@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data with no nulls or duplicates"
)
def customer_silver():
    return dlt.read("customer_bronze")
        .dropDuplicates(["CustId"])
        .filter(col("CustId").isNotNull() &
                col("Name").isNotNull() &
                col("EmailId").isNotNull() &
                col("Region").isNotNull())

@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return dlt.read("order_bronze")
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))

        .dropDuplicates(["OrderId"])
        .filter(col("OrderId").isNotNull() &
                col("ItemName").isNotNull() &
                col("PricePerUnit").isNotNull() &
                col("Qty").isNotNull() &
                col("Date").isNotNull() &
                col("CustId").isNotNull())

```

### ## Step 3: Creating the Order Summary Table with SCD Type 2

This step involves creating an initial order summary table with SCD Type 2 columns.

```

@dlt.table(
    name="ordersummary",
    comment="Order summary with SCD Type 2 tracking",
    table_properties={
        "quality": "gold",
        "delta.enableChangeDataFeed": "true"
    }
)
@dlt.expect_or_fail("valid_join_key", "CustId IS NOT NULL")
def ordersummary():
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")

    joined_df = customer_df.join(
        order_df,
        on="CustId",
        how="inner"
    ).select(
        customer_df["CustId"],
        customer_df["Name"],
        customer_df["EmailId"],
        customer_df["Region"],
        order_df["OrderId"],
        order_df["ItemName"],
        order_df["PricePerUnit"],

```

```

        order_df["Qty"],
        order_df["Date"],
        order_df["TotalAmount"]
    )

    result_df = joined_df.withColumn("IsActive", lit(True))
        .withColumn("StartDate", current_timestamp().cast("date"))
        .withColumn("EndDate", lit(None).cast(DateType()))

    return result_df

```

#### ## Step 4: Implementing SCD Type 2 Update Logic

The code defines a temporary table `ordersummary\_updates` to hold the current state of the data and then applies SCD Type 2 logic to the `ordersummary\_scd2` table.

```

@dlt.table(
    name="ordersummary_updates",
    comment="Updates to the order summary table",
    temporary=True
)
def ordersummary_updates():
    customer_df = dlt.read("customer_silver")
    order_df = dlt.read("order_silver")

    current_df = customer_df.join(
        order_df,
        on="CustId",
        how="inner"
    ).select(
        customer_df["CustId"],
        customer_df["Name"],
        customer_df["EmailId"],
        customer_df["Region"],
        order_df["OrderId"],
        order_df["ItemName"],
        order_df["PricePerUnit"],
        order_df["Qty"],
        order_df["Date"],
        order_df["TotalAmount"]
    )

    return current_df

@dlt.table(
    name="ordersummary_scd2",
    comment="SCD Type 2 implementation of order summary",
    table_properties={
        "quality": "gold",
        "delta.enableChangeDataFeed": "true"
    }
)
def ordersummary_scd2():
    from delta.tables import DeltaTable
    import datetime

```

```

current_data = dlt.read("ordersummary_updates")

try:
    existing_table = dlt.read("ordersummary")

    changed_records = current_data.join(
        existing_table.filter(col("IsActive") == True),
        on=["CustId", "OrderId"],
        how="inner"
    ).where(
        (current_data["Name"] != existing_table["Name"]) |
        (current_data["EmailId"] != existing_table["EmailId"]) |
        (current_data["Region"] != existing_table["Region"]) |
        (current_data["ItemName"] != existing_table["ItemName"])
        |
        (current_data["PricePerUnit"] != existing_table["PricePe
rUnit"]) |
        (current_data["Qty"] != existing_table["Qty"]) |
        (current_data["Date"] != existing_table["Date"]) |
        (current_data["TotalAmount"] != existing_table["TotalAmo
unt"])
    ).select(
        existing_table["CustId"],
        existing_table["OrderId"]
    )

    updated_existing = existing_table.join(
        changed_records,
        on=["CustId", "OrderId"],
        how="left_outer"
    ).withColumn(
        "IsActive",
        when(changed_records["CustId"].isNotNull(), False).other
wise(existing_table["IsActive"])
    ).withColumn(
        "EndDate",
        when(changed_records["CustId"].isNotNull(), current_time
stamp().cast("date")).otherwise(existing_table["EndDate"])
    )

    new_records = current_data.join(
        changed_records,
        on=["CustId", "OrderId"],
        how="inner"
    ).withColumn("IsActive", lit(True))
    .withColumn("StartDate", current_timestamp().cast("date"))
    .withColumn("EndDate", lit(None).cast(DateType()))

    completely_new = current_data.join(
        existing_table.select("CustId", "OrderId"),
        on=["CustId", "OrderId"],
        how="left_anti"
    ).withColumn("IsActive", lit(True))
    .withColumn("StartDate", current_timestamp().cast("date"))
    .withColumn("EndDate", lit(None).cast(DateType()))

    return updated_existing.union(new_records).union(completely_
new)
except:

```

```
        return current_data.withColumn("IsActive", lit(True))
                                .withColumn("StartDate", current_timestamp().cast("date")
))
                                .withColumn("EndDate", lit(None).cast(DateType()))
```

## Step 5: Creating the Customer Aggregate Spend Table

The final step involves creating a table that aggregates customer spending by name and date.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date"
)
def customeraggregatespend():
    order_summary = dlt.read("ordersummary_scd2")

    return order_summary.filter(col("IsActive") == True)
                            .groupBy("Name", "Date")
                            .agg({"TotalAmount": "sum"})
                            .withColumnRenamed("sum(TotalAmount)", "TotalAmount")
```